

SISTEMAS OPERATIVOS

AGENTIC OS V.2.0

Diagrama de Despliegue y Reporte

Estudiante: Mario Alberto Alvarado Giler
Paralelo: 1

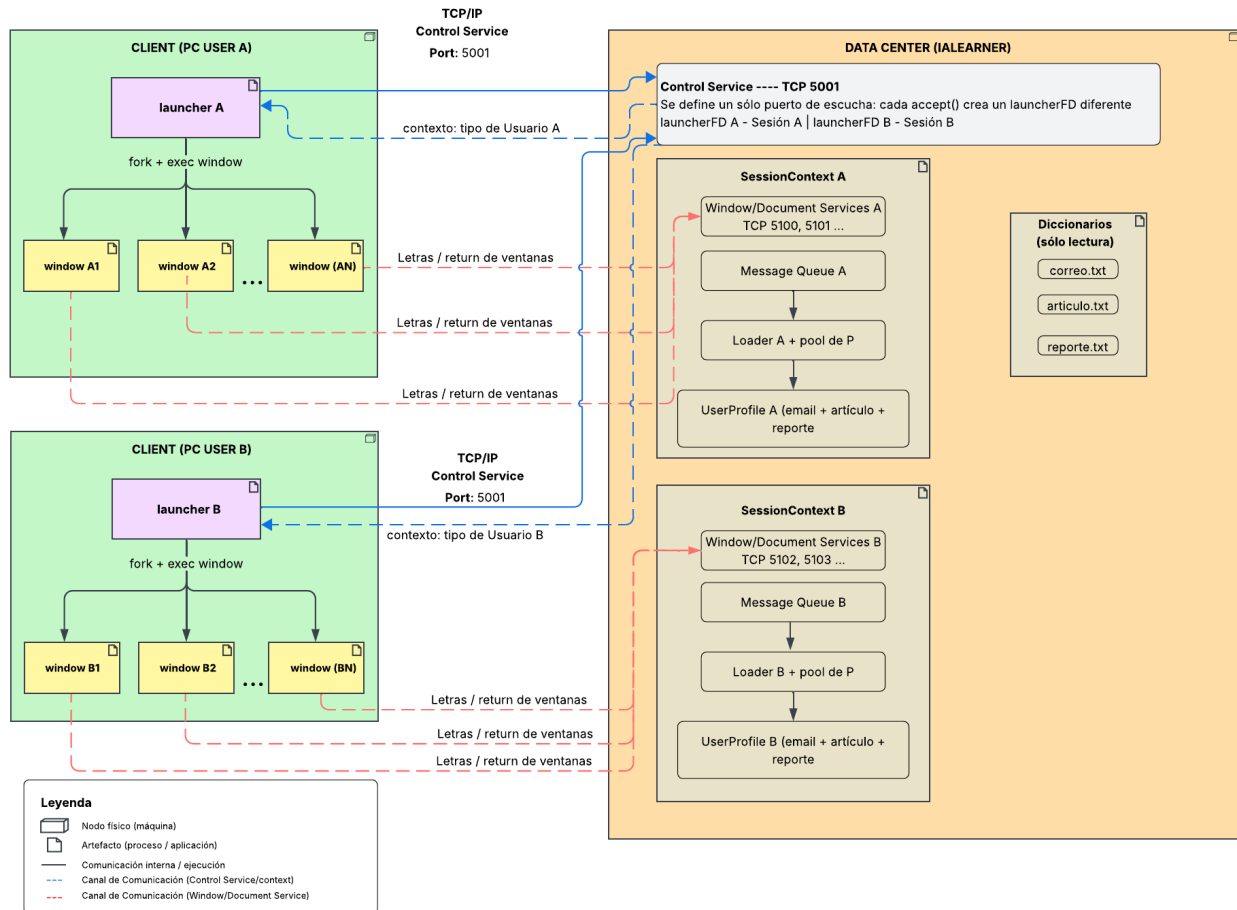


Fig. 1 Diagrama de Despliegue de AGENTIC OS V.2.0

La solución propuesta para **AGENTIC OS V.2.0** se construyó siguiendo una **arquitectura cliente-servidor** orientada a separar claramente la administración local de procesos, la captura de información y el procesamiento concurrente en el data center. Como se observa en el diagrama de despliegue, cada computadora de usuario ejecuta una instancia independiente del launcher, cuya responsabilidad es crear, monitorear y cerrar únicamente sus procesos gráficos locales. Para crear cada ventana se utilizan las llamadas fork() y exec(), de manera de que cada window se ejecuta como un proceso autónomo y puede ser supervisado individualmente mediante su PID. El launcher dispone de un canal TCP/IP de control con IALearner a través del puerto 5001; este canal se emplea para iniciar y finalizar la sesión, solicitar los servicios necesarios para las ventanas y recibir el contexto inferido, es decir, el tipo de usuario correspondiente. El launcher no transporta las letras introducidas por el usuario: cada ventana captura directamente los eventos del teclado mediante X11 y envía los caracteres, incluido return, al servicio remoto de documentos que IALearner asignó para ella.

Aunque todos los launchers se conectan al mismo puerto de control 5001, sus sesiones no se mezclan. Este puerto funciona solamente como punto común de escucha. Cada llamada a accept() produce un descriptor de conexión diferente, denominado launcherFD, que permite a IALearner distinguir al launcher A del launcher B, incluso cuando ambos utilizan la misma dirección y el

mismo puerto de destino. Por cada conexión aceptada se crea un hilo de atención y un SessionContext independiente. Esta estructura reúne el socket del launcher, los puertos y servicios de sus ventanas, la cola de oraciones, el Loader, el pool de detectores y el perfil documental acumulado. De esta manera, SessionContext A contiene exclusivamente la información generada por las ventanas A1, A2 hasta AN, mientras que SessionContext B administra las ventanas B1, B2 hasta BN. Esta decisión arquitectónica permite conservar la integridad de los datos y evita que una oración o clasificación producida por un usuario modifique el perfil de otro.

Cuando un launcher solicita la creación de una ventana, IALearner selecciona un puerto de datos disponible, comenzando en el 5100, y crea un servicio de documentos asociado con el SessionContext que originó la solicitud. El launcher recibe el puerto asignado y lo entrega al nuevo proceso window. A partir de ese momento, la ventana establece directamente su conexión TCP con IALearner y envía un carácter por cada tecla válida presionada. El hilo remoto que atiende esa conexión acumula las letras en búfer dinámico propio, por lo que varias ventanas pueden escribir simultáneamente sin compartir sus datos parciales. Al recibirse el carácter return, el contenido acumulado se considera una oración completa, se copia en la cola concurrente de la sesión correspondiente y el búfer queda disponible para formar la siguiente oración. Así, la formación de oraciones ocurre de manera concurrente y asíncrona respecto de las demás ventanas.

Para administrar las oraciones se implementó una cola circular dinámica protegida mediante un mutex y una variable de condición. El mutex garantiza que varias ventanas puedan insertar elementos y que el Loader pueda extraerlos sin provocar inconsistencias. La variable de condición permite suspender al Loader mientras no existan suficientes oraciones, evitando ciclos de espera activa y consumo innecesario de CPU. El parámetro P, recibido al iniciar IALearner, determina tanto la cantidad de oraciones necesarias para activar un lote como el número máximo de detectores que pueden trabajar simultáneamente. Antes de utilizarlo, el programa valida que sea positivo y lo limita de acuerdo con la cantidad de CPU disponibles. La distribución efectiva de los workers entre los procesadores queda a cargo del planificador del sistema operativo.

La metodología de concurrencia utiliza un pool permanente de P hilos detectores por sesión. Estos hilos se crean una sola vez y permanecen suspendidos sobre una variable de condición mientras no tienen tareas asignadas. Cuando la cola alcanza P oraciones, el Loader extrae el lote, asigna una oración a cada worker y despierta a los detectores. Cada hilo procesa exactamente una oración, por lo que hasta P clasificaciones pueden ejecutarse en paralelo. Al terminar, los workers notifican la finalización del lote y vuelven a quedar suspendidos, listos para ser reutilizados. Este enfoque evita crear y destruir hilos repetidamente, reduce el costo de administración y satisface el límite de concurrencia establecido. Si llegan más de P oraciones, las adicionales permanecen en la cola mientras se procesa el lote activo; posteriormente, el Loader reactiva el pool con el siguiente grupo. Al cerrar una sesión también se procesa el remanente inferior a P, evitando perder oraciones pendientes.

La clasificación aplica la técnica *bag of words*. Cada oración se transforma a minúsculas y se tokeniza con strtok_r, cuya naturaleza reentrante permite que varios detectores trabajen simultáneamente sin compartir el estado de tokenización. Los tokens se consultan en tres diccionarios almacenados como tablas hash de solo lectura: correo electrónico, artículo científico y reporte. Una clase documental se considera válida cuando la oración contiene al menos tres palabras de su diccionario. Si una oración satisface más de una clase, se selecciona aquella con la mayor cantidad de coincidencias. Los diccionarios se cargan una sola vez en el contexto general del servidor y pueden ser consultados concurrentemente, mientras que los perfiles y colas permanecen separados por sesión.

Apenas un detector obtiene la clase de su oración, registra el resultado en el UserProfile de su propio SessionContext y evalúa inmediatamente la tabla de decisión. Esta actualización está protegida por un mutex, pero no requiere esperar que todos los detectores del lote terminen; por ello, la determinación del usuario es asíncrona respecto de las demás clasificaciones. La combinación de documentos permite identificar personal administrativo, personal técnico, profesor o estudiante. Durante la ejecución, IALearner muestra la clase, las coincidencias obtenidas y el tipo de usuario vigente. Al finalizar la sesión procesa las oraciones pendientes y envía el tipo definitivo únicamente por el launcherFD asociado, garantizando que cada launcher reciba el contexto producido por sus propias ventanas.

Finalmente, la implementación aplica programación defensiva mediante la validación de parámetros, memoria, sockets, procesos e hilos. Las secciones críticas se mantienen pequeñas y se protegen con mutex específicos para la cola, el lote, el perfil, la sesión y la salida de consola. Las variables de condición mantienen suspendidos al Loader y a los workers cuando no existe trabajo. Durante el cierre se detienen los servicios de ventana, se procesan los elementos pendientes, se despiertan y unen los hilos, se cierran los descriptores y se liberan las estructuras dinámicas. En conjunto, esta metodología integra multiprogramación, IPC con sockets TCP, concurrencia con Pthreads, sincronización y TDAs apropiadas, manteniendo aislado el contexto de cada usuario y aprovechando eficientemente los recursos disponibles.